Project: Mini-Project

Milan Lagae

Institution/University Name: Vrije Universiteit Brussel
Faculty: Sciences and Bioengineering Sciences
Course: GPU Computing

Contents

1 Intro

Throughout all parts of the mini-project, it was chosen to compare two **operations** namely: addition & multiplication. To start, a comparison between the operations will be done, as requested in part 1. Following that, the kernel will be updated to execute on multiple elements simultaneous (one-to-many) mapping.

For the third part, the compute intensity will be increased by applying a loop count factor over the computations.

The final part will compare the local vs global memory location, as indicated in the scheme[1] in the assignment.

2 Structure

In the zip file, inside of the src, there is a project-desktop folder, containing the c++ files which execute the kernel benchmarks. For each part, there is a separate c++ file. The kernel files for each part, are located in the root src folder. The following list of kernels should be present in the root folder: partOne, partTwo, partThree, partFour.

3 Methodology

For each combination of values to be measures, **20** runs where measured. The first 5 runs where discarded to allow the system to stabilize. This is in accordance to the recommendations in [1]. For each measured value, if applicable a Cov range chart is produced and will be reference, these can be found in §8. The acceptable range of the Cov is

[1] http://parallel.vub.ac.be/education/gpu/labs/miniproject_step4_with_local%20memory.jpg

taken from [2]. The Cov charts for each part, can be found in the appendix section §8.

4 Part 1: Addition vs Multiplication

This section will discuss, comparing the performance between the addition and multiplication operations. The general gpu speedup compared to cpu will also be discussed.

4.1 Setup

The code for this part can be found in the file: part-1.cpp and the kernel file in partOne.cl Included in the partOne.cl kernel file, are two separate kernels, respectively: mul_continuous , add_continuous.

4.2 GPU Analysis

Let's start of with analyzing the gpu performance. The easiest chart to get started, is time chart. Plotting the time vs the array size can be seen in Figure 1. With the color, differentiating between the 2 operations.

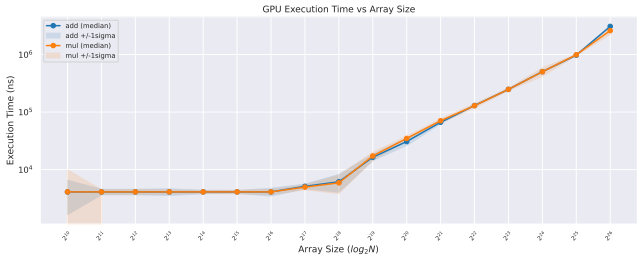

Figure 1:

The only notable, to remark on the plot, is that only starting from an array size of 2^{16}, does the computational intensity increase enough for the work to be reflected in the time taking. Starting from this point one, the time logarithmically with the size of the array.

Continuing from the time variable, to the memory bandwidth vs array size in Figure 2.

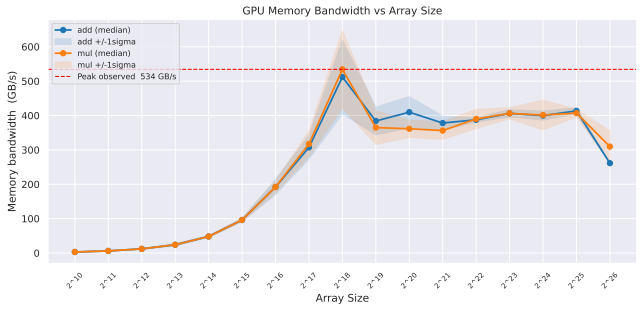

Figure 2:

The memory bandwidth increases with the increase in array size, with a peak at the array size of 2^{18}.

Increasing the size of the array beyond this 'ridge point', we are limited by the kernel (construction inefficiency) and available memory bandwidth.Increasing the array size to 2^{26} and beyond, showcase that the gpu **may** potentially be memory limited.

Before making strong conclusion's, let's take a look at the computationally intensity of the benchmark in Figure 3.

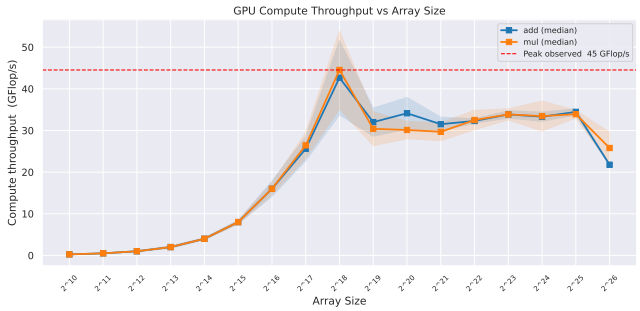

Figure 3:

The same curve as in Figure 2 is visible. The initial conclusion that can be drown from this figure and the previous one, is that at the moment, this kernel is memory bound.

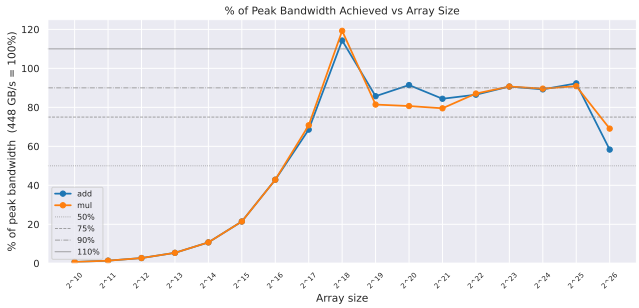

Figure 4:

When plotting the memory bandwidth in percentage to the max theoretical performance, it is clearly visible that the GPU is memory bound. The peak at array size of 2^{18}, could be explained by the fact the two source arrays and target arrays can be stored inside of the gpu's L2 cache, which for an RTX 3070 is 4MB[2].

4.3 CPU vs GPU

Let's start by comparing the GPU performance to CPU performance, this graph is depicted in Figure 5.

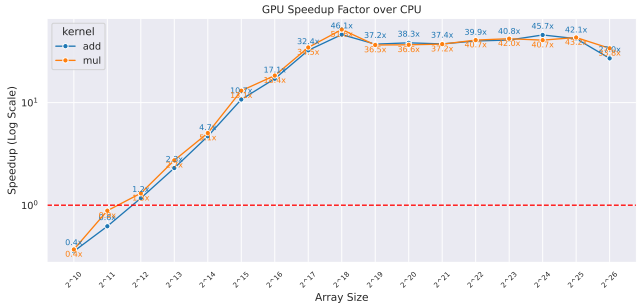

Figure 5:

Comparison to CPU performance is done using, a very naive CPU algorithm which does a loop over the loop. A better comparison, would be the usage of parallelism & potential even SIMD to further increase CPU performance. For this part, the CPU will keep using a naive algorithm.

As is visible on figure Figure 5, the speed gained by using the parallelism of the GPU has a massive impact on the performance between both.

4.4 Conclusion

From the conduced benchmarks and resulting graphs, it can be concluded that there is not considerably different in speed between the additional or multiplication operation on a RTX 3070 gpu. The comparison of the GPU to a naive CPU algorithm, showcase the massive parallelism capabilities of GPU's.

[2] https://www.techpowerup.com/gpu-specs/geforce-rtx-3070.c3674

5 Part 2: Elements Per Thread

This section will analyze the result of increasing the number of elements a single thread (kernel item) is responsible for on the performance of the execution. These different access patterns are described as continuous & strided. They are based on the pdf[3] mentioned in the assignment description.

5.1 Setup

The array size is fixed for all benchmark variations performed in this part to 2^{22}, based on the data gathered in part 1 and the workgroup size is set to 64.

The benchmark file is called: part-2.cpp and kernel file partTwo.cl. Included in the kernel file are the following kernels: mul_contiguous, add_continguous, mul_strided, add_strided.

The contiguous pattern, corresponds t the example code shown in Listing 1.

```
1 // host code
2 unsigned int data_index[W];
3 cl::NDRange global(W/N);
4 // device code
5 for (int i = 0; i < N; ++i)
6     data_index[N * get_global_id(0) + i] =
  get_global_id(0);
```

Listing 1:

The strided access pattern, corresponds to the example code shown in Listing 2.

```
1 // host code
2 unsigned int data_index[W];
3 cl::NDRange global(W/N);
4
5 // device code
6 for (int i = 0; i < N; ++i)
7     data_index[get_global_id(0) + i*get_global_size(0)] =
  get_global_id(0);
```

Listing 2:

5.2 Visualization

TODO

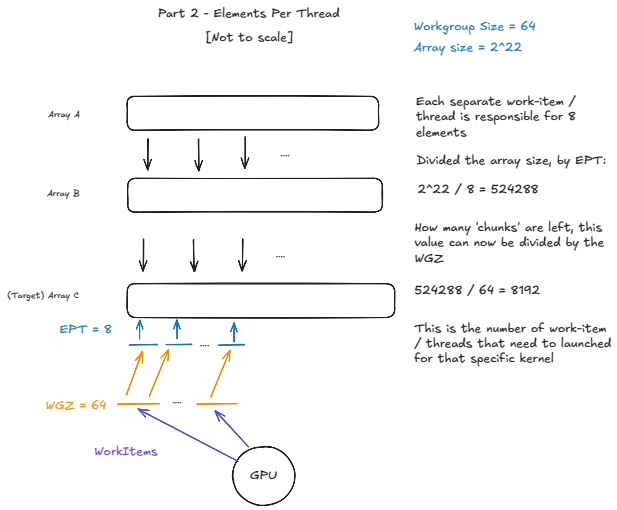

Figure 6:

5.3 Continuous vs Strided

Start of with looking at the time it takes between both operations and different access patterns in figure Figure 7.

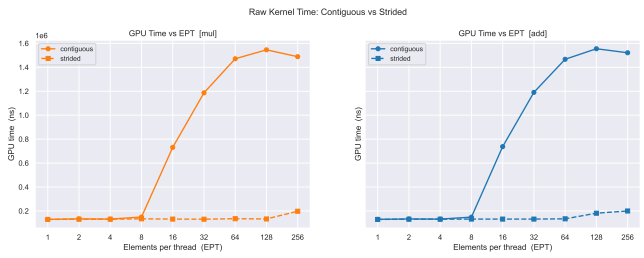

Figure 7:

[3] http://parallel.vub.ac.be/education/gpu/theory/GPU%20Computing%20-%20Lesson%202%20doc%20-%20Programming%20GPUs%20-%20levels%200%201%20and%202.pdf

This chart gives the first indication that the strided access pattern has better performance compared to the continuous pattern.

When putting the continuous pattern vs the strided pattern, and plotting the speedup in figure Figure 17, the picture becomes clearer.

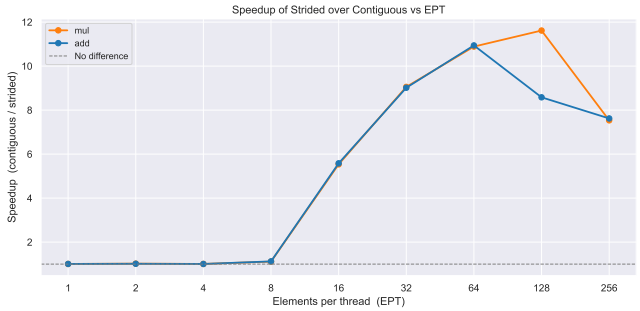

Figure 8:

The graphs show a very clear speedup of the strided access pattern starting from 8 ETP value. The reason for this speedup of this pattern becomes clear when looking at the memory bandwidth for the patterns in Figure 9.

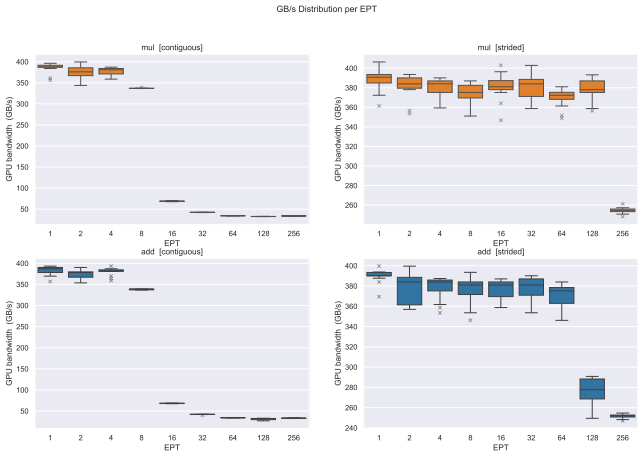

Figure 9:

The continuous access pattern is shown on the left and the strided on the right. The large drop in memory bandwidth for the continuous pattern when the EPT value is increased from 8 to 16 is noticeable. A similar drop in bandwidth is remarked for the continuous pattern when going from 64 to 128 and to 256.

Let's continue to take a look at the compute intensity of both strategies, as shown in Figure 10.

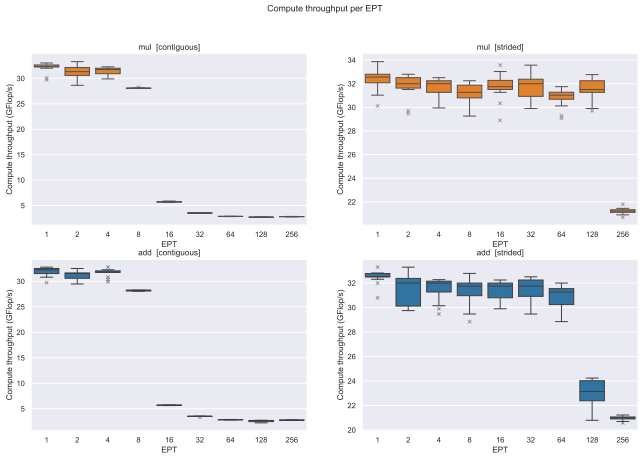

Figure 10:

Here the picture becomes much clearer again, at the same time that the memory bandwidth takes a sharp drop, the compute throughput does to. The drop for the continuous pattern arrives at the same time, and for the strided pattern also.

But comparing the add vs mul operation, the drop for the mul operation appears to be delayed by one EPT value increase.

5.4 Conclusion

Combining all the information from the charts above, it can be concluded that the strided pattern gives the GPU the data it needs to perform the computations, up until the elements per thread become to large again and the gpu is unable to saturate the bandwidth and give the gpu the data it needs to compute.

Apart from the performance that can be noticed from the data, is the fact that the CI interval of the continuous are much tighter than those of the strided pattern. No immediate answer can be given for this difference in intervals between access patterns.

6 Part 3: Roofline Model

This section will discuss the experiment to create a roofline model as described in the assignment & discussed in the lectures. Before showcasing the roofline model, several other charts will be discussed first, showcasing the utility of the roofline model chart.

6.1 Setup

The array size is fixed for all benchmark variations performed in this part to 2^{22}, based on the data gathered in part 1 and the workgroup size is set to 64. The benchmark file is called part-3.cpp and kernel file: partThree.cl. Included in the kernel file is a single kernel called: float_sum_increasing_ci. It combines the elements per thread loop with the kernel intSumIncreasingCI, present in the list of gpu exercises given at the start of the semester. The benchmark file, also update the formulas for calculating the bandwidth (throughput), compute intensity (flops), and arithmetic intensity with formulas from the sumIntsIncreasingCI.cpp file.

6.2 Analysis

Let's start with some charts, depicting the loop count for the compute intensity, memory bandwidth and runtime as show in Figure 11.

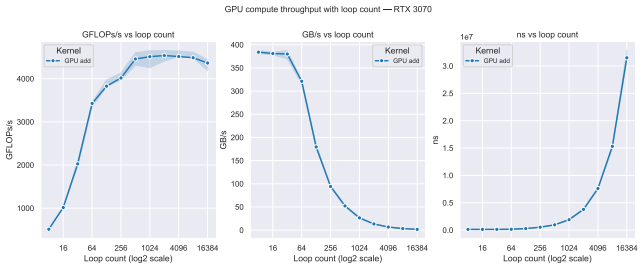

Figure 11:

The plots on the graphs do not indicated out of the ordinary, increasing the loop count increases the runtime of the execution. The behavior between the compute intensity (left) and memory bandwidth (middle) is interesting and warrants further discussion, as designed by the experiment.

Before analyzing the roofline model, let's compare the compute intensity for the different loop count values in function of the elements per thread, in figure Figure 12.

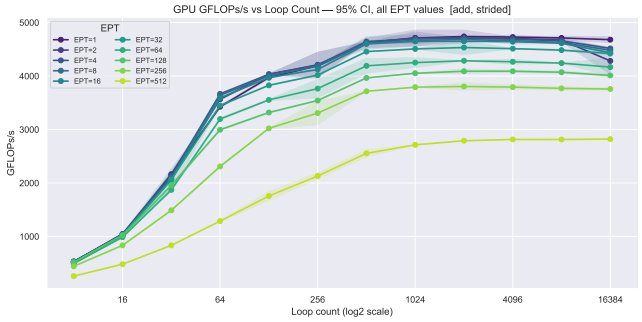

Figure 12:

A similar graph can be made for the bandwidth in function of the loop count and different EPT values, in Figure 13.

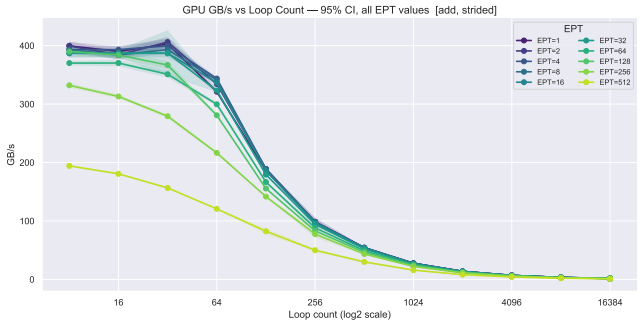

Figure 13:

The same behavior as seen in graphs in figure Figure 11, is again visible in the above two graphs, for different loop counts. The EPT value 'only' determines the ranges of the values, but does not change the chart behavior.

Plotting the arithmetic intensity, in function of the loop count as shown in Figure 14.

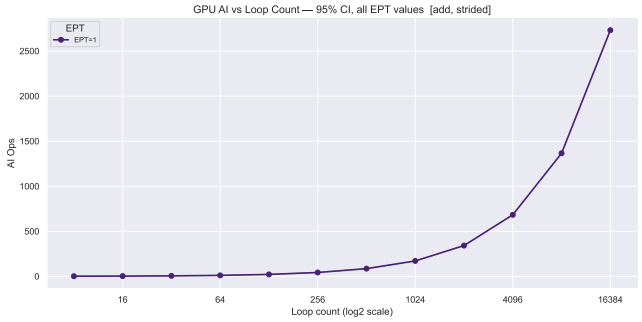

Figure 14:

The model, as seen in the lectures, which summaries all the plots and behavior's described is the roofline model, as shown in Figure 15.

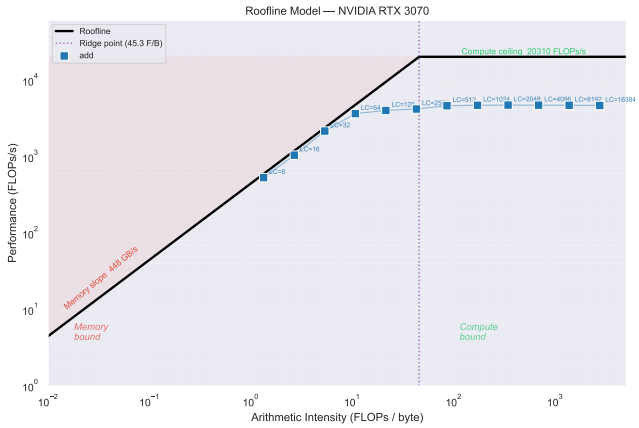

Figure 15:

For the GPU in question, the memory bandwidth & peak computational throughput is indicated. The results follow the memory sloop from the starting loop count (LC) 8 until 64, were it starts diverging. The 'ridge point' at which the actual data start becoming 'memory bound' occurs much earlier than the theoretical value.

6.3 Conclusion

The experiment in question, has clearly shown the appearance of the roofline model in the data, this part of the experiment can be considered a success. With small note made for the practical ridge point not matching the expected theoretical point.

7 Part 4: Local vs Global

7.1 Setup

8 Appendix

8.1 Part 1

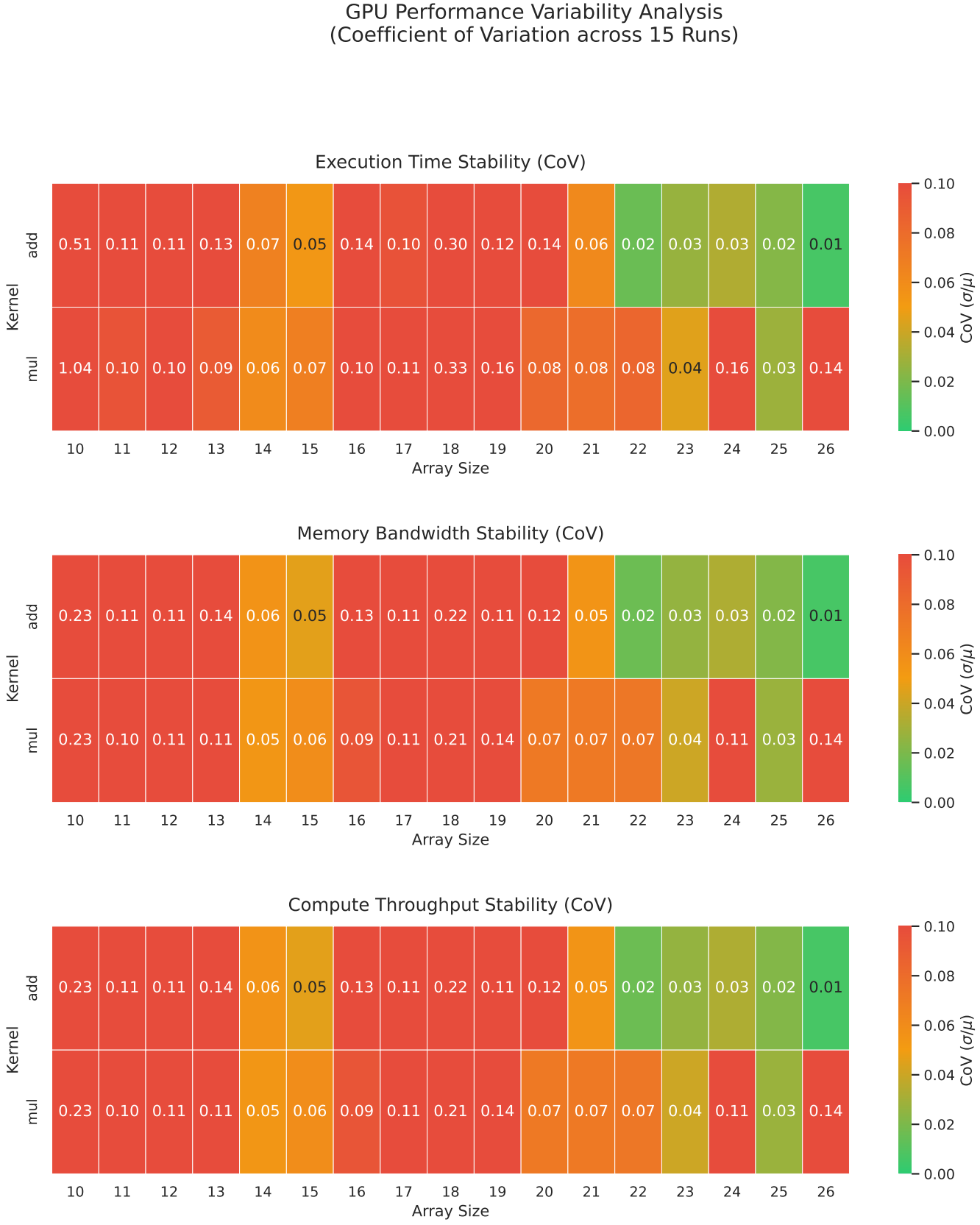
GPU Performance Variability Analysis
(Coefficient of Variation across 15 Runs)
Execution Time Stability (CoV)
Kernel
add
mul
0.51 0.11 0.11 0.13 0.07 0.05 0.14 0.10 0.30 0.12 0.14 0.06 0.02 0.03 0.03 0.02 0.01
1.04 0.10 0.10 0.09 0.06 0.07 0.10 0.11 0.33 0.16 0.08 0.08 0.08 0.04 0.16 0.03 0.14
10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26
Array Size
0.10
0.08
0.06
0.04
0.02
0.00
CoV (σ/μ)
Memory Bandwidth Stability (CoV)
Kernel
add
mul
0.23 0.11 0.11 0.14 0.06 0.05 0.13 0.11 0.22 0.11 0.12 0.05 0.02 0.03 0.03 0.02 0.01
0.23 0.10 0.11 0.11 0.05 0.06 0.09 0.11 0.21 0.14 0.07 0.07 0.07 0.04 0.11 0.03 0.14
10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26
Array Size
0.10
0.08
0.06
0.04
0.02
0.00
CoV (σ/μ)
Compute Throughput Stability (CoV)
Kernel
add
mul
0.23 0.11 0.11 0.14 0.06 0.05 0.13 0.11 0.22 0.11 0.12 0.05 0.02 0.03 0.03 0.02 0.01
0.23 0.10 0.11 0.11 0.05 0.06 0.09 0.11 0.21 0.14 0.07 0.07 0.07 0.04 0.11 0.03 0.14
10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26
Array Size
0.10
0.08
0.06
0.04
0.02
0.00
CoV (σ/μ)

Figure 16:

8.2 Part 2

Figure 17:

8.3 Part 3

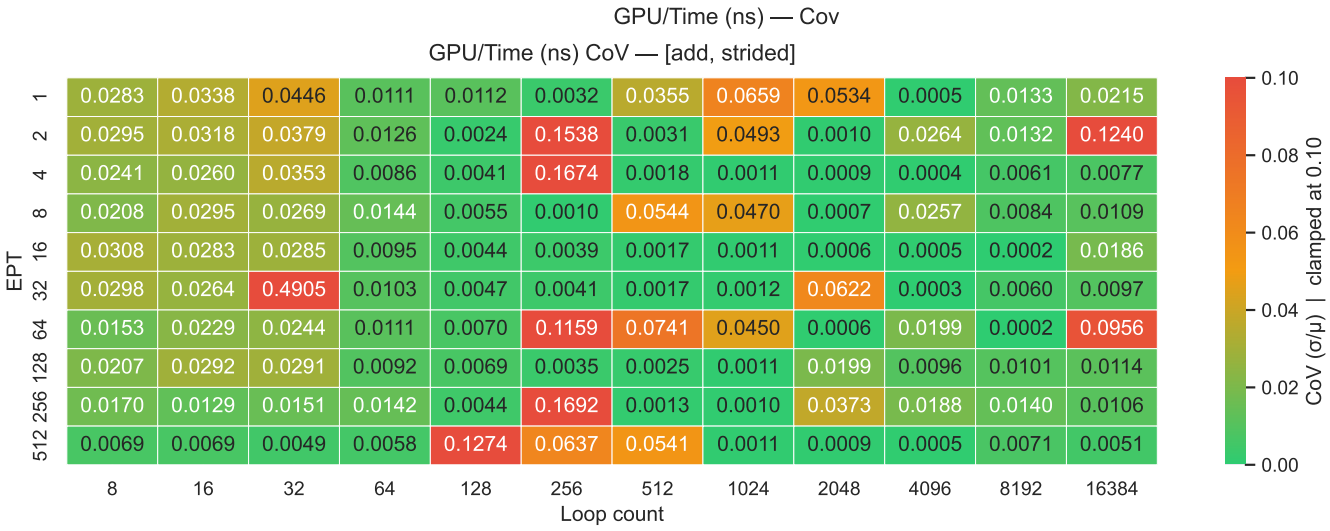

Figure 18:

8.4 GPU Microbenchmark

8.4.1 Desktop

NVIDIA GeForce RTX 3070 (platform NVIDIA CUDA)

Expert level: Expert (3)

microbenchmarks measuring computational performance				
measure all	Perf (GOps)	I (c)	L (c)	r (warps)
SP	6370	0.42	2.77	128
SP/2 indep	7460	0.358	1.33	64
SP/4 indep	8190	0.326	0.803	160
SP/8 indep	4400	0.607	1.06	24
MADD	13600	0.197	1.16	128
MADD/2 indep	15900	0.168	0.683	64
MADD/4 indep	17400	0.153	0.517	160
INT	4460	0.599	2.33	128
INT/2 indep	4880	0.547	1.33	64
INT/4 indep	5140	0.519	1.33	64
INT/8 indep	1340	1.99	2.59	8
IMADD	8830	0.303	1.26	128
IMADD/2 indep	9680	0.276	0.674	64
IMADD/4 indep	10300	0.259	0.466	64
DIV	3050	0.876	3.36	64
DIV/2 indep	3280	0.814	2.51	96
DIV/4 indep	3350	0.798	1.47	48
DIV/8 indep	3330	0.813	1.09	12
DP	37.8	70.6	79.4	3
DP/2 indep	38	70.5	78.1	2
DP/4 indep	38.3	69.8	71.5	2
DP/8 indep	...	...	...	...
SF	686	3.89	10.7	32
SF/2 indep	1400	1.9	2.97	24
SF/4 indep	2770	0.964	1.39	16
SFSOFT	214	12.6	44.4	24
SFSOFT/2 indep	256	10.5	38.8	160
SFSOFT/4 indep	261	10.3	44.4	24
ATAN	...	...	...	...
CMP	2530	1.06	5.61	64
CMP/2 indep	2630	1.01	1.83	32
MOV	...	...	...	...
KERNEL	1510	4.77	4.77	...
GLOBALID	185	14.5	510	320
LOCALID	204	13.1	435	352

Figure 19: Desktop - Microbenchmark Results 1

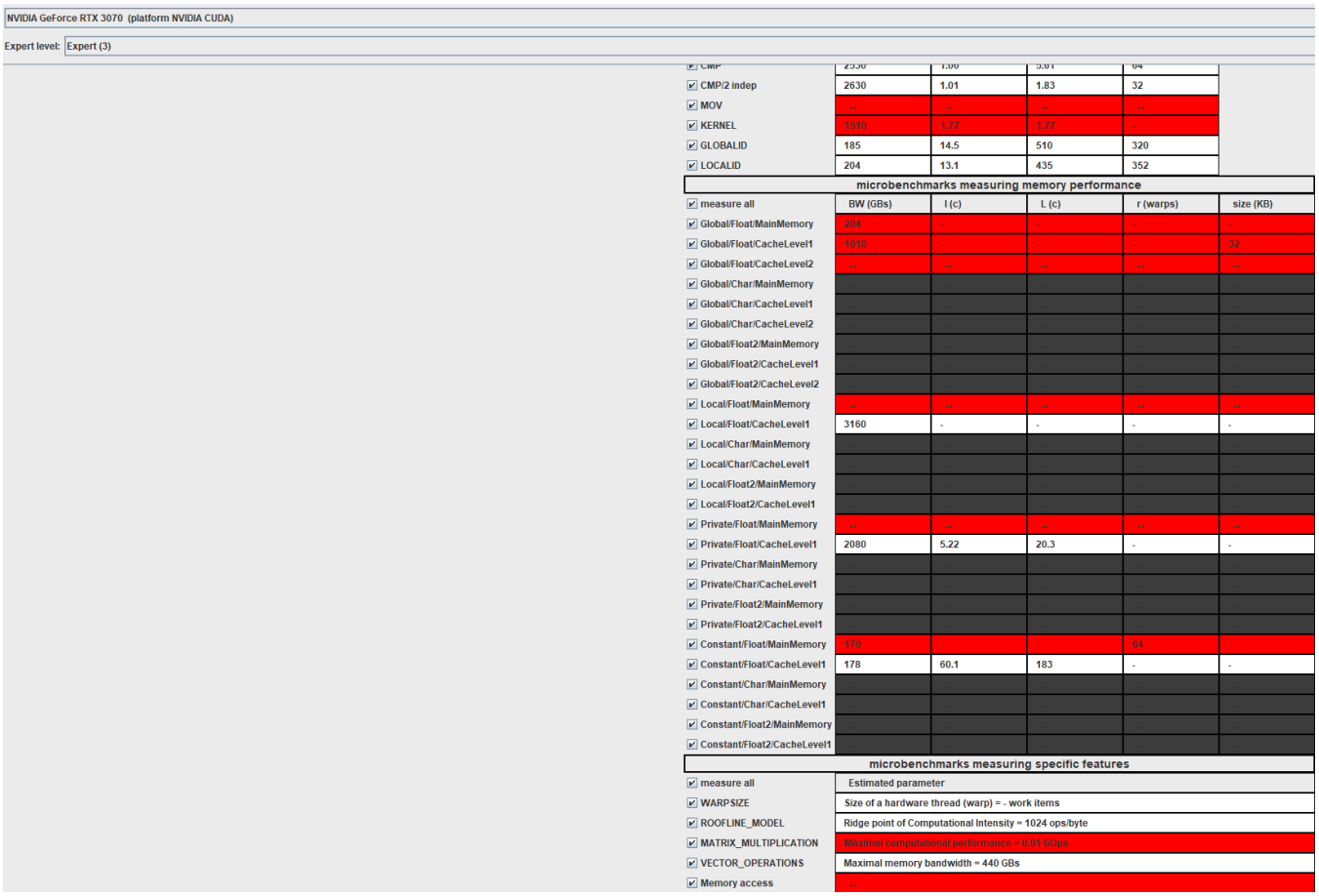

Figure 20: Desktop - Microbenchmark Results 2

8.5 Specifications

8.5.1 Macbook

Part	Value
CPU	M2 Pro (6 performance and 4 efficiency)
OpenCL	1.2
RAM	16GB
OS	**TODO**

Table 1: Macbook Specifications

8.5.2 Desktop

Part	Value
CPU	Ryzen 9 5950X
GPU	RTX 3070
OpenCL	300 (Set in code)
Driver Version	591.59
RAM	64GB (3200 Mhz)
OS	Windows 11 - Version 10.0.22631 Build 22631

Table 2: Desktop Specifications

Bibliography

[1] P. A. Paolillo, “Lecture 2 - Metrics, Distributions & Experimental Rigor - How Many runs do I need?.” p. 38, 2025.
[2] P. A. Paolillo, “Lecture 2 - Metrics, Distributions & Experimental Rigor - Coefficient of Variation (CoV).” p. 25, 2025.